

Universidad de Costa Rica

Laboratorio de Microcontroladores

Laboratorio #3:
Arduino: GPIO, ADC y comunicaciones

Prof. MSc. Marco Villalta Fallas

Marco Vásquez Ovares
B17032
Grupo: 01

I Ciclo 2025

Índice

1. Introducción	5
2. Nota teórica	6
2.1. Arduino Uno	6
2.1.1. Diagrama de pines	7
2.1.2. Descripción de pines	7
2.1.3. Características eléctricas	8
2.2. Microcontrolador ATmega328P	9
2.2.1. Diagrama de pines	10
2.2.2. Descripción de pines	11
2.2.3. Diagrama de bloques	14
2.2.4. Registros de trabajo de propósito general de la CPU AVR	15
2.2.5. Características eléctricas	15
2.3. Pantalla PCD8544	17
2.3.1. Diagrama de pines	18
2.3.2. Diagrama de bloques	19
2.3.3. Descripción de pines	20
2.3.4. Características eléctricas	20
2.4. Comunicación UART/USART	23
2.5. Protocolo de Comunicación SPI (Serial Peripheral Interface)	24
2.5.1. Arquitectura del SPI	24
2.6. Diseño del circuito	25
2.6.1. Lista de componentes	26
2.6.2. Información adicional	26
3. Desarrollo	28
3.1. Análisis del firmware	28
3.1.1. Inclusión de Librerías y Declaración de Variables	28
3.1.2. Función setup()	28
3.1.3. Función de Interrupción: clear_display()	29
3.1.4. Funciones de Visualización	29
3.1.5. Modo de Medición AC	30
3.1.6. Análisis detallado de la función get_RMS()	30
3.1.7. Modo de Medición DC	32
3.1.8. Advertencia de Alto Voltaje	32
3.1.9. Selector de Modo	32
3.1.10. Bucle Principal	32
3.2. Análisis electrónico	33
3.2.1. Conversión de rango de tensión: De [-24V, +24V] a [0V, +5V]	33
3.2.2. Implementación de los interruptores (Switches)	35
3.2.3. Iluminación de los Leds	35
3.2.4. Pantalla PCD8544	36
3.2.5. Señales de los 4 canales	37
3.3. Análisis de resultados	39
3.3.1. Análisis en modo AC	39

3.3.2.	Análisis en modo DC	40
3.3.3.	Archivo .csv	42
4.	Conclusiones y Recomendaciones	43
4.1.	Conclusiones	43
4.2.	Recomendaciones	43
	Bibliografía	44
5.	Apéndices	44
5.1.	Repositorio Git	44
5.2.	Archivo .sh	45
5.3.	Archivo .py	46

Índice de figuras

1.	The Arduino UNO R3	6
2.	Connector Pinouts	7
3.	JANALOG	7
4.	JDIGITAL	8
5.	Power Consumption	8
6.	Pinout 1	10
7.	Pinout 2	11
8.	Port Pin Configurations	11
9.	Port B Pins Alternate Functions	12
10.	Port C Pins Alternate Functions	12
11.	Port D Pins Alternate Functions	13
12.	Diagrama de bloques	14
13.	AVR CPU General Purpose Working Registers	15
14.	Absolute Maximum Ratings	15
15.	DC Characteristics	15
16.	DC Characteristics (Continued)	16
17.	DC Characteristics	16
18.	Pantalla LCD Gráfica PCD8544 (Nokia 5110)	18
19.	Diagrama de pines	18
20.	Diagrama de bloques	19
21.	Descripción de pines	20
22.	VALORES LÍMITE	20
23.	DC CHARACTERISTICS	21
24.	DC CHARACTERISTICS	22
25.	AC CHARACTERISTICS	22
26.	Diseño final, parte 1	25
27.	Diseño final, parte 2	25
28.	Divisor de tensión con dos fuentes	33
29.	Conversión de rango de tensión	34
30.	Interruptores	35
31.	Led de advertencia	36
32.	Funcionamiento de la pantalla PCD8544	37
33.	Visualización de las señales a través del osciloscopio	38
34.	Resultados de las mediciones AC.	39
35.	Funcionamiento en modo AC y USART activado	40
36.	Resultados de las mediciones DC.	41
37.	Funcionamiento en modo DC y USART activado	41
38.	Archivo .csv con las mediciones del multímetro	42

Índice de tablas

1.	Lista de componentes	26
----	--------------------------------	----

1. Introducción

Resumen

Este laboratorio tuvo como propósito el diseño e implementación de un multímetro digital de 4 canales, utilizando como plataforma principal la placa Arduino Uno junto con una pantalla LCD PCD8544 y una interfaz de comunicación serial hacia una computadora. El sistema fue desarrollado en el entorno de simulación SimulIDE, permitiendo verificar el funcionamiento sin requerir hardware físico.

El objetivo funcional del sistema fue la medición de voltajes analógicos en el rango de -24V a +24V en cuatro entradas independientes, utilizando los canales analógicos del microcontrolador ATmega328P. Para esto, se aplicó una técnica de conversión basada en el convertidor analógico a digital (ADC) interno de 10 bits, que entrega valores entre 0 y 1023, con un punto medio (referencia) ubicado en 511.5, representando el cero lógico (0V).

El sistema trabaja en dos modos de operación:

- **Modo DC** (corriente continua): Se mide directamente la señal presente en cada canal, calculando su valor de voltaje en base al desplazamiento desde el punto medio.
- **Modo AC (corriente alterna)**: Se realiza un muestreo intensivo de 1000 lecturas por canal para encontrar el pico máximo de la señal alterna, que luego es convertido en un valor RMS (Root Mean Square) usando un factor de corrección de 0.7071, típico para ondas senoidales.

La selección entre modo DC y modo AC se realiza mediante la lectura del pin analógico A5, funcionando como selector de modo según su voltaje. Para cada modo, el sistema muestra los resultados en la pantalla PCD8544 en formato organizado, con leyendas como "V1: 10.23V" para cada canal. Además, los datos se envían por el puerto UART/USART hacia una computadora para su registro, utilizando un script en Python que almacena la información en un archivo CSV.

Un aspecto importante del sistema fue la implementación de LEDs de advertencia, uno por cada canal. Estos LEDs están conectados a los pines digitales 8, 9, 10 y 11, y se encienden automáticamente si el voltaje medido en ese canal supera los $\pm 20V$, tanto en modo AC como DC. Esta funcionalidad permite una alerta visual inmediata ante condiciones peligrosas o fuera del rango operativo seguro.

También se integró una interrupción externa en el pin digital 2, que permite limpiar la pantalla cuando se detecta un cambio lógico en ese pin, agregando una funcionalidad adicional para mejorar la experiencia del usuario.

En conjunto, el laboratorio permitió integrar conocimientos de lectura analógica, manejo de pantallas gráficas, comunicación serial, uso de interrupciones y diseño de sistemas de advertencia visual, demostrando la versatilidad del Arduino Uno en aplicaciones de instrumentación electrónica básica.

2. Nota teórica

2.1. Arduino Uno

El **Arduino Uno** es una placa de desarrollo basada en el microcontrolador **ATmega328P**, diseñada para facilitar el aprendizaje y la implementación de sistemas electrónicos y de control. Es una de las placas más populares de la plataforma Arduino debido a su simplicidad, bajo costo y gran comunidad de soporte.

Cuenta con:

- 14 pines digitales de entrada/salida (de los cuales 6 pueden ser usados como salidas PWM)
- 6 entradas analógicas (A0 a A5)
- Un oscilador de 16 MHz
- Un conector USB tipo B para programación y comunicación.
- Un conector de alimentación externa.
- Un botón de reinicio (reset).

El Arduino Uno se programa utilizando el lenguaje de programación C/C++ simplificado a través del IDE de Arduino, lo que permite cargar instrucciones directamente al microcontrolador mediante USB. Gracias a su arquitectura abierta, es ideal para proyectos educativos, prototipado rápido y automatización básica.



Figura 1: The Arduino UNO R3

2.1.1. Diagrama de pines

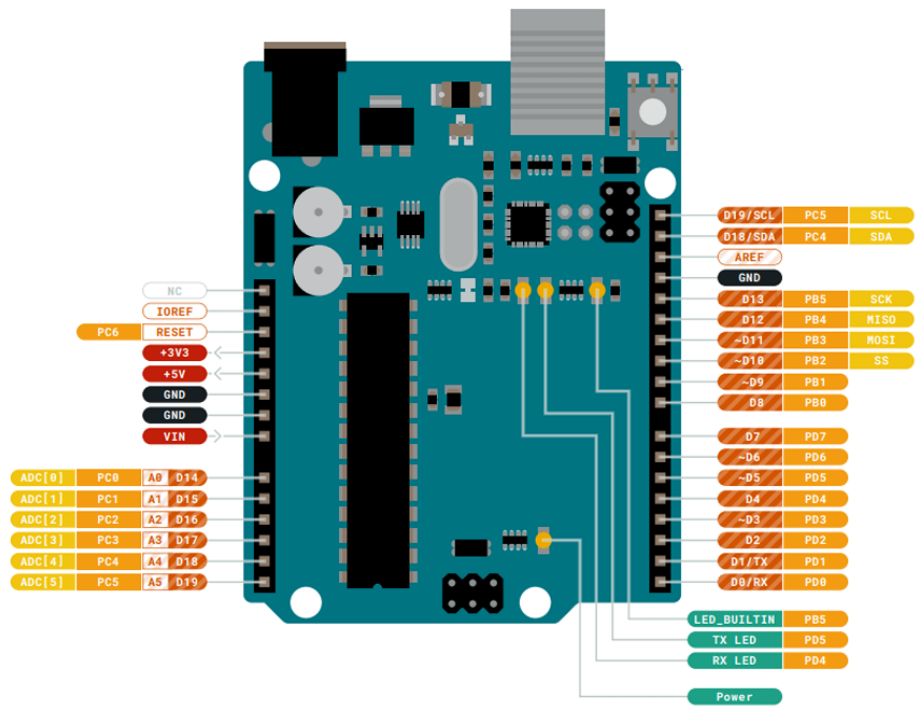


Figura 2: Connector Pinouts

2.1.2. Descripción de pines

Pin	Function	Type	Description
1	NC	NC	Not connected
2	IOREF	IOREF	Reference for digital logic V - connected to 5V
3	Reset	Reset	Reset
4	+3V3	Power	+3V3 Power Rail
5	+5V	Power	+5V Power Rail
6	GND	Power	Ground
7	GND	Power	Ground
8	VIN	Power	Voltage Input
9	A0	Analog/GPIO	Analog input 0 /GPIO
10	A1	Analog/GPIO	Analog input 1 /GPIO
11	A2	Analog/GPIO	Analog input 2 /GPIO
12	A3	Analog/GPIO	Analog input 3 /GPIO
13	A4/SDA	Analog input/I2C	Analog input 4/I2C Data line
14	A5/SCL	Analog input/I2C	Analog input 5/I2C Clock line

Figura 3: JANALOG

Pin	Function	Type	Description
1	D0	Digital/GPIO	Digital pin 0/GPIO
2	D1	Digital/GPIO	Digital pin 1/GPIO
3	D2	Digital/GPIO	Digital pin 2/GPIO
4	D3	Digital/GPIO	Digital pin 3/GPIO
5	D4	Digital/GPIO	Digital pin 4/GPIO
6	D5	Digital/GPIO	Digital pin 5/GPIO
7	D6	Digital/GPIO	Digital pin 6/GPIO
8	D7	Digital/GPIO	Digital pin 7/GPIO
9	D8	Digital/GPIO	Digital pin 8/GPIO
10	D9	Digital/GPIO	Digital pin 9/GPIO
11	SS	Digital	SPI Chip Select
12	MOSI	Digital	SPI1 Main Out Secondary In
13	MISO	Digital	SPI Main In Secondary Out
14	SCK	Digital	SPI serial clock output
15	GND	Power	Ground
16	AREF	Digital	Analog reference voltage
17	A4/SD4	Digital	Analog input 4/I2C Data line (duplicated)
18	A5/SD5	Digital	Analog input 5/I2C Clock line (duplicated)

Figura 4: JDIGITAL

2.1.3. Características eléctricas

Symbol	Description	Min	Typ	Max	Unit
VINMax	Maximum input voltage from VIN pad	6	-	20	V
VUSBMax	Maximum input voltage from USB connector		-	5.5	V
PMax	Maximum Power Consumption	-	-	xx	mA

Figura 5: Power Consumption

2.2. Microcontrolador ATmega328P

El ATmega328P es un microcontrolador de 8 bits desarrollado por Microchip Technology (anteriormente Atmel). Es ampliamente utilizado en sistemas embebidos y forma parte central de placas como el Arduino Uno. Su balance entre simplicidad, eficiencia energética y capacidad de procesamiento lo hacen ideal para aplicaciones educativas, industriales y de prototipado rápido.

■ Arquitectura y Núcleo

- Arquitectura: AVR RISC de 8 bits
- Núcleo: AVR Enhanced RISC
- Conjunto de instrucciones eficiente, con la mayoría de instrucciones ejecutadas en un solo ciclo de reloj
- Frecuencia de operación: hasta 20 MHz (usualmente 16 MHz en Arduino Uno)

■ Memoria

- 32 KB de memoria Flash (almacena el programa)
- 1 KB de EEPROM (almacena datos no volátiles)
- 2 KB de SRAM (memoria de datos en tiempo de ejecución)

■ Entradas y Salidas (I/O)

- 23 líneas de E/S (I/O) programables
- Pines digitales y analógicos multipropósito
- 6 canales de entrada analógica (ADC) con resolución de 10 bits
- 3 timers/counters:
 - Dos de 8 bits
 - Uno de 16 bits
- Soporte para PWM (modulación por ancho de pulso) en múltiples pines

■ Periféricos Integrados

- Convertidor ADC de 10 bits, hasta 15kSPS
- USART, para comunicación serie síncrona/asíncrona (UART)
- SPI (Serial Peripheral Interface)
- I2C (TWI – Two Wire Interface)
- Watchdog Timer
- Comparador analógico
- Soporte para interrupciones externas e internas

■ Voltaje y Consumo

- Voltaje de operación: 1.8V – 5.5V
 - Bajo consumo energético en modos de reposo (sleep)
 - Múltiples modos de ahorro de energía: Idle, ADC Noise Reduction, Power-down, Power-save, Standby, Extended Standby
- Encapsulado y Dimensiones
- Encapsulado común: DIP de 28 pines (ideal para protoboard)
 - También disponible en SMD: TQFP, QFN

2.2.1. Diagrama de pines

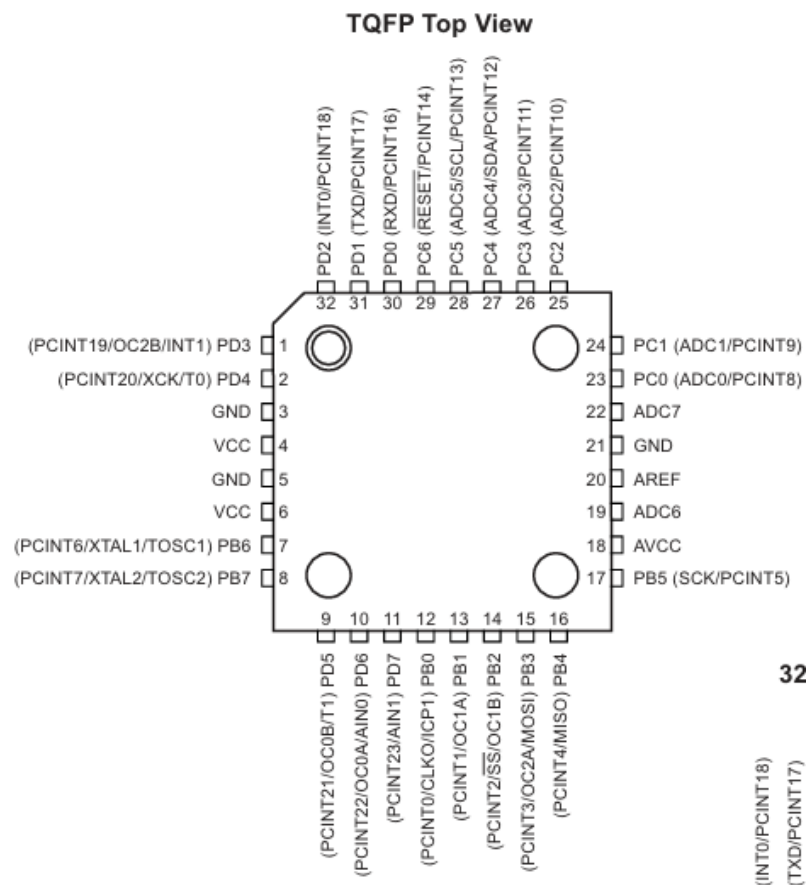


Figura 6: Pinout 1

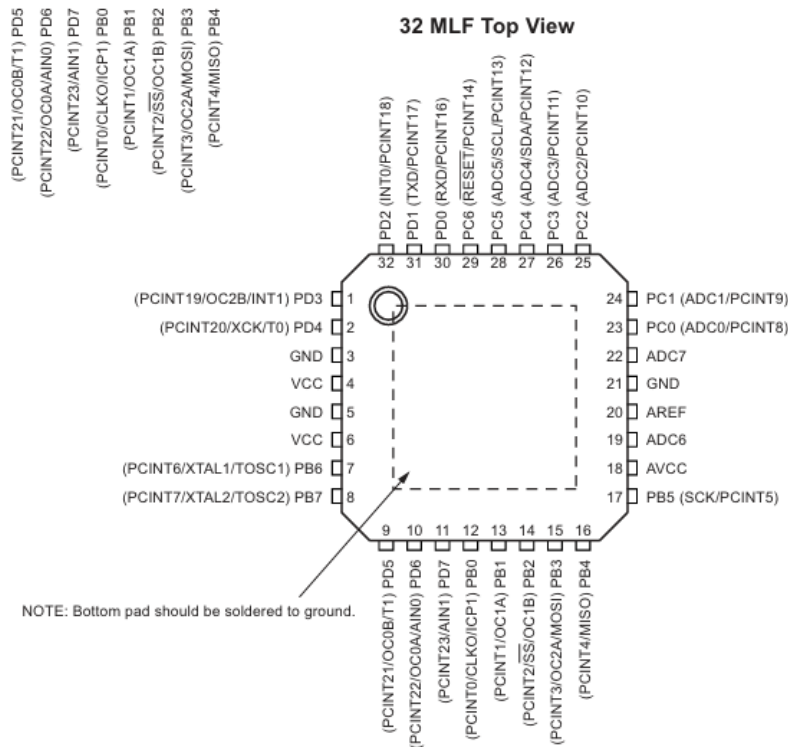


Figura 7: Pinout 2

2.2.2. Descripción de pines

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output low (sink)
1	1	X	Output	No	Output high (source)

Figura 8: Port Pin Configurations

Alternate Port Functions

Port Pin	Alternate Functions
PB7	XTAL2 (chip clock oscillator pin 2) TOSC2 (timer oscillator pin 2) PCINT7 (pin change interrupt 7)
PB6	XTAL1 (chip clock oscillator pin 1 or external clock input) TOSC1 (timer oscillator pin 1) PCINT6 (pin change interrupt 6)
PB5	SCK (SPI bus master clock input) PCINT5 (pin change interrupt 5)
PB4	MISO (SPI bus master input/slave output) PCINT4 (pin change interrupt 4)
PB3	MOSI (SPI bus master output/slave input) OC2A (Timer/Counter2 output compare match A output) PCINT3 (pin change interrupt 3)
PB2	$\overline{SS}$ (SPI bus master slave select) OC1B (Timer/Counter1 output compare match B output) PCINT2 (pin change interrupt 2)
PB1	OC1A (Timer/Counter1 output compare match A output) PCINT1 (pin change interrupt 1)
PB0	ICP1 (Timer/Counter1 input capture input) CLKO (divided system clock output) PCINT0 (pin change interrupt 0)

Figura 9: Port B Pins Alternate Functions

Port Pin	Alternate Function
PC6	$\overline{RESET}$ (reset pin) PCINT14 (pin change interrupt 14)
PC5	ADC5 (ADC input channel 5) SCL (2-wire serial bus clock line) PCINT13 (pin change interrupt 13)
PC4	ADC4 (ADC input channel 4) SDA (2-wire serial bus data input/output line) PCINT12 (pin change interrupt 12)
PC3	ADC3 (ADC input channel 3) PCINT11 (pin change interrupt 11)
PC2	ADC2 (ADC input channel 2) PCINT10 (pin change interrupt 10)
PC1	ADC1 (ADC input channel 1) PCINT9 (pin change interrupt 9)
PC0	ADC0 (ADC input channel 0) PCINT8 (pin change interrupt 8)

Figura 10: Port C Pins Alternate Functions

Port Pin	Alternate Function
PD7	AIN1 (analog comparator negative input) PCINT23 (Pin Change Interrupt 23)
PD6	AIN0 (analog comparator positive input) OC0A (Timer/Counter0 output compare match A output) PCINT22 (pin change interrupt 22)
PD5	T1 (Timer/Counter 1 external counter input) OC0B (Timer/Counter0 output compare match B output) PCINT21 (pin change interrupt 21)
PD4	XCK (USART external clock input/output) T0 (Timer/Counter 0 external counter input) PCINT20 (pin change interrupt 20)
PD3	INT1 (external interrupt 1 input) OC2B (Timer/Counter2 output compare match B output) PCINT19 (pin change interrupt 19)
PD2	INT0 (external interrupt 0 input) PCINT18 (pin change interrupt 18)
PD1	TXD (USART output pin) PCINT17 (pin change interrupt 17)
PD0	RXD (USART input pin) PCINT16 (pin change interrupt 16)

Figura 11: Port D Pins Alternate Functions

2.2.3. Diagrama de bloques

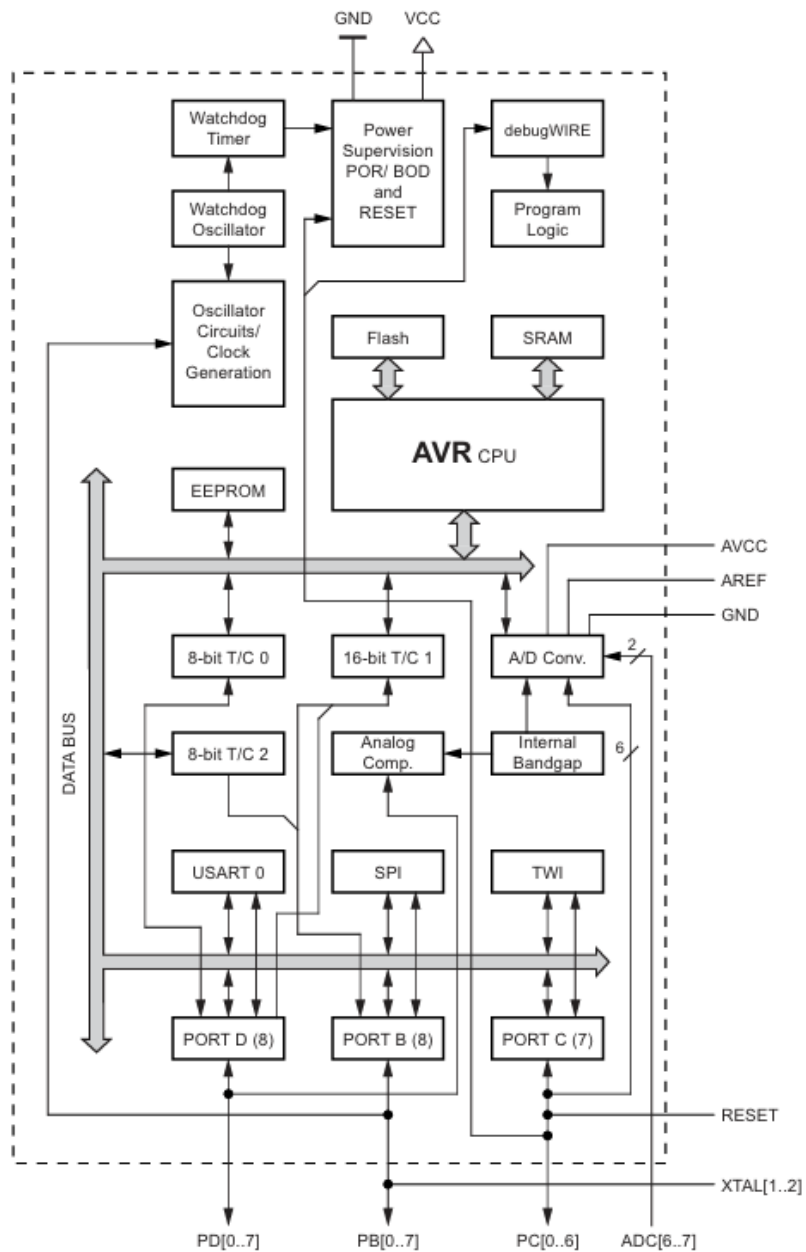


Figura 12: Diagrama de bloques

2.2.4. Registros de trabajo de propósito general de la CPU AVR

General Purpose Working Registers	7	0	Addr.	
	R0		0x00	
	R1		0x01	
	R2		0x02	
	...			
	R13		0x0D	
	R14		0x0E	
	R15		0x0F	
	R16		0x10	
	R17		0x11	
	...			
	R26		0x1A	X-register Low Byte
	R27		0x1B	X-register High Byte
	R28		0x1C	Y-register Low Byte
	R29		0x1D	Y-register High Byte
	R30		0x1E	Z-register Low Byte
	R31		0x1F	Z-register High Byte

Figura 13: AVR CPU General Purpose Working Registers

2.2.5. Características eléctricas

Parameters	Min.	Typ.	Max.	Unit
Operating temperature	-55		+125	°C
Storage temperature	-65		+150	°C
Voltage on any pin except RESET with respect to ground	-0.5		$V_{CC} + 0.5$	V
Voltage on RESET with respect to ground	-0.5		+13.0	V
Maximum operating voltage		6.0		V
DC current per I/O pin		40.0		mA
DC current V_{CC} and GND pins		200.0		mA
Injection current at $V_{CC} = 0V$		$\pm 5.0^{(1)}$		mA
Injection current at $V_{CC} = 5V$		± 1.0		mA

Note: 1. Maximum current per port = $\pm 30mA$

Figura 14: Absolute Maximum Ratings

$T_A = -40^\circ C$ to $+125^\circ C$, $V_{CC} = 2.7V$ to $5.5V$ (unless otherwise noted)

Parameter	Condition	Symbol	Min.	Typ.	Max.	Unit
Input low voltage, except XTAL1 and RESET pin	$V_{CC} = 2.7V$ to $5.5V$	V_{IL}	-0.5		$0.3V_{CC}^{(1)}$	V
Input high voltage, except XTAL1 and RESET pins	$V_{CC} = 2.7V$ to $5.5V$	V_{IH}	$0.6V_{CC}^{(2)}$		$V_{CC} + 0.5$	V
Input low voltage, XTAL1 pin	$V_{CC} = 2.7V$ to $5.5V$	V_{IL1}	-0.5		$0.1V_{CC}^{(1)}$	V
Input high voltage, XTAL1 pin	$V_{CC} = 2.7V$ to $5.5V$	V_{IH1}	$0.7V_{CC}^{(2)}$		$V_{CC} + 0.5$	V

Figura 15: DC Characteristics

$T_A = -40^\circ\text{C}$ to $+125^\circ\text{C}$, $V_{CC} = 2.7\text{V}$ to 5.5V (unless otherwise noted)

Parameter	Condition	Symbol	Min.	Typ.	Max.	Unit
Input low voltage, RESET pin	$V_{CC} = 2.7\text{V}$ to 5.5V	V_{IL2}	-0.5		$0.1V_{CC}^{(1)}$	V
Input high voltage, RESET pin	$V_{CC} = 2.7\text{V}$ to 5.5V	V_{IH2}	$0.9V_{CC}^{(2)}$		$V_{CC} + 0.5$	V
Output low voltage ⁽³⁾	$I_{OL} = 20\text{mA}$, $V_{CC} = 5\text{V}$ $I_{OL} = 5\text{mA}$, $V_{CC} = 3\text{V}$	V_{OL}			0.8 0.5	V
Output high voltage ⁽⁴⁾	$I_{OH} = -20\text{mA}$, $V_{CC} = 5\text{V}$ $I_{OH} = -10\text{mA}$, $V_{CC} = 3\text{V}$	V_{OH}	4.1 2.3			V
Input leakage Current I/O pin	$V_{CC} = 5.5\text{V}$, pin low (absolute value)	I_{IL}			1	μA
Input leakage Current I/O pin	$V_{CC} = 5.5\text{V}$, pin high (absolute value)	I_{IH}			1	μA
Reset pull-up resistor		R_{RST}	30		60	$\text{k}\Omega$
I/O pin pull-up resistor		R_{PU}	20		50	$\text{k}\Omega$
Analog comparator Input offset voltage	$0.4\text{V} < V_{in} < V_{CC} - 0.5$ (absolute value)	V_{ACIO}		10	40	mV
Analog comparator Input leakage current	$V_{CC} = 5\text{V}$ $V_{in} = V_{CC}/2$	I_{ACLK}	-50		+50	nA

Figura 16: DC Characteristics (Continued)

$T_A = -40^\circ\text{C}$ to $+125^\circ\text{C}$, $V_{CC} = 2.7\text{V}$ to 5.5V (unless otherwise noted)

Parameter	Condition	Symbol	Min.	Typ. ⁽²⁾	Max.	Units
Power supply current ⁽¹⁾	Active 4MHz, $V_{CC} = 3\text{V}$	I_{CC}		1.5	2.4	mA
	Active 8MHz, $V_{CC} = 5\text{V}$			5.2	10	mA
	Active 16MHz, $V_{CC} = 5\text{V}$			9.2	14	mA
	Idle 4MHz, $V_{CC} = 3\text{V}$			0.25	0.6	mA
	Idle 8MHz, $V_{CC} = 5\text{V}$			1.0	1.6	mA
	Idle 16MHz, $V_{CC} = 5\text{V}$			1.9	2.8	mA
Power-down mode ⁽³⁾	WDT enabled, $V_{CC} = 3\text{V}$	I_{CC}			44	μA
	WDT enabled, $V_{CC} = 5\text{V}$				66	μA
	WDT disabled, $V_{CC} = 3\text{V}$				40	μA
	WDT disabled, $V_{CC} = 5\text{V}$				60	μA

Figura 17: DC Characteristics

2.3. Pantalla PCD8544

La pantalla PCD8544, también conocida como pantalla Nokia 5110, es un módulo LCD gráfico de bajo consumo ampliamente utilizado en proyectos de electrónica y microcontroladores debido a su bajo costo, facilidad de uso y eficiencia energética.

Características generales:

- Controlador: Philips PCD8544
- Resolución: 84 x 48 píxeles (gráfica monocromática)
- Tecnología: LCD (pantalla de cristal líquido pasiva con retroiluminación LED opcional)
- Modo de visualización: Blanco y negro (1 bit por píxel)
- Interfaz: Serial tipo SPI
- Voltaje de operación:
 - Lógica: 2.7V a 3.3V (no tolera 5V en sus entradas)
 - Retroiluminación: típicamente alimentada con una resistencia limitadora desde 3.3V o 5V
- Consumo: Muy bajo (200 μ A en operación, menor en reposo)
- Pines del Módulo
 - RST – Reset del controlador (activo en bajo)
 - CE (CS) – Chip Enable o Chip Select (activo en bajo)
 - DC – Selección de datos o comandos
 - DIN (MOSI) – Datos de entrada (SPI)
 - CLK (SCLK) – Reloj de entrada para SPI
 - VCC – Alimentación (3.3V)
 - BL – Retroiluminación LED (puede conectarse a GND con una resistencia)
 - GND – Tierra
- Funcionamiento y Control
 - Utiliza una interfaz SPI de 4 hilos, lo que permite una comunicación eficiente con microcontroladores como el ATmega328P
 - Requiere una librería específica (como PCD8544, Adafruit_PCD8544 o U8g2) para facilitar el dibujo de texto, gráficos y píxeles.
 - El controlador almacena los datos en una memoria interna (buffer RAM), por lo que los píxeles deben enviarse y actualizarse manualmente.

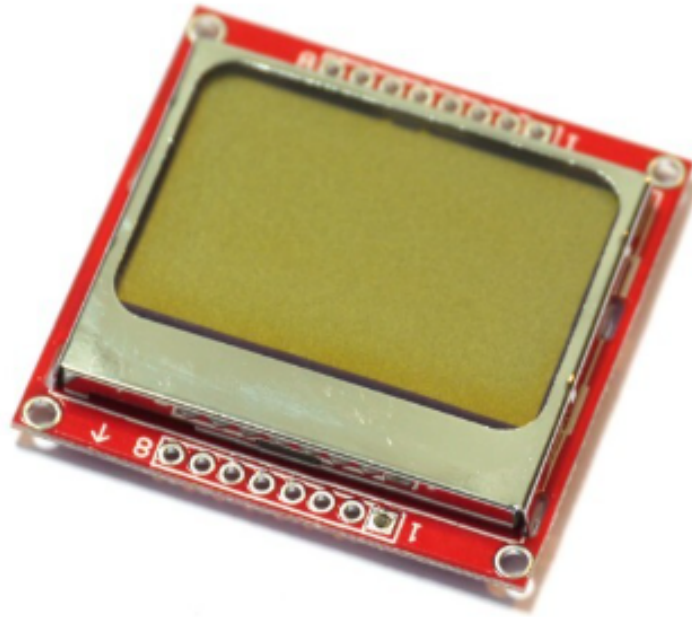


Figura 18: Pantalla LCD Gráfica PCD8544 (Nokia 5110)

2.3.1. Diagrama de pines

Display Pin	Arduino Pin
Pin 1	+3.3V Pin
Pin 2 (SCLK)	Digital Pin 3
Pin 3 (SDIN/MOSI)	Digital Pin 4
Pin 4 (D/C)	Digital Pin 5
Pin 5 (SCE)	Digital Pin 7
Pin 6	Ground Pin
Pin 7	10uF capacitor to Ground Pin
Pin 8 (RST)	Digital Pin 6

Figura 19: Diagrama de pines
<https://github.com/carlosefr/pcd8544>

2.3.2. Diagrama de bloques

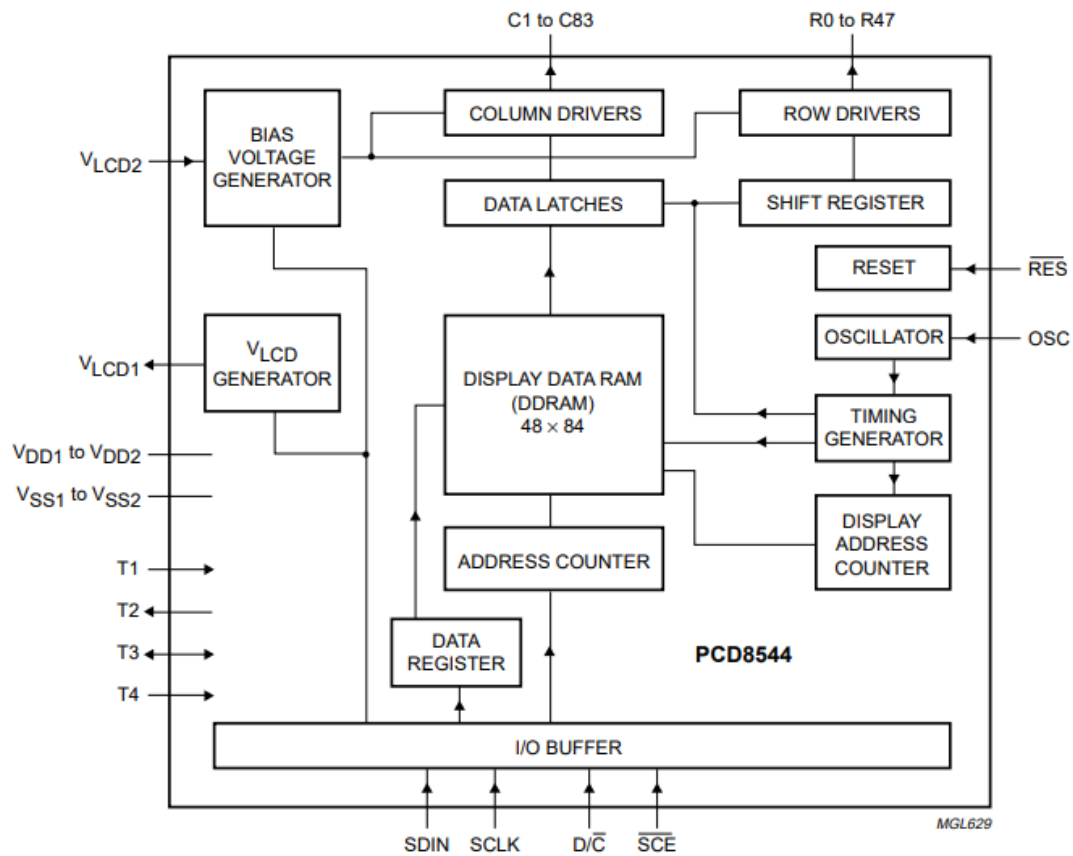


Figura 20: Diagrama de bloques

2.3.3. Descripción de pines

SYMBOL	DESCRIPTION
R0 to R47	LCD row driver outputs
C0 to C83	LCD column driver outputs
V _{SS1} , V _{SS2}	ground
V _{DD1} , V _{DD2}	supply voltage
V _{LCD1} , V _{LCD2}	LCD supply voltage
T1	test 1 input
T2	test 2 output
T3	test 3 input/output
T4	test 4 input
SDIN	serial data input
SCLK	serial clock input
D/ $\overline{C}$	data/command
$\overline{SCE}$	chip enable
OSC	oscillator
$\overline{RES}$	external reset input
dummy1, 2, 3, 4	not connected

Figura 21: Descripción de pines

2.3.4. Características eléctricas

SYMBOL	PARAMETER	CONDITIONS	MIN.	MAX.	UNIT
V _{DD}	supply voltage	note 3	-0.5	+7	V
V _{LCD}	supply voltage LCD	note 4	-0.5	+10	V
V _i	all input voltages		-0.5	V _{DD} + 0.5	V
I _{SS}	ground supply current		-50	+50	mA
I _i , I _O	DC input or output current		-10	+10	mA
P _{tot}	total power dissipation		-	300	mW
P _O	power dissipation per output		-	30	mW
T _{amb}	operating ambient temperature		-25	+70	°C
T _j	operating junction temperature		-65	+150	°C
T _{stg}	storage temperature		-65	+150	°C

Figura 22: VALORES LÍMITE

$V_{DD} = 2.7$ to 3.3 V; $V_{SS} = 0$ V; $V_{LCD} = 6.0$ to 9.0 V; $T_{amb} = -25$ to $+70$ °C; unless otherwise specified.

SYMBOL	PARAMETER	CONDITIONS	MIN.	TYP.	MAX.	UNIT
V_{DD1}	supply voltage 1	LCD voltage externally supplied (voltage generator disabled)	2.7	–	3.3	V
V_{DD2}	supply voltage 2	LCD voltage internally generated (voltage generator enabled)	2.7	–	3.3	V
V_{LCD1}	LCD supply voltage	LCD voltage externally supplied (voltage generator disabled)	6.0	–	9.0	V
V_{LCD2}	LCD supply voltage	LCD voltage internally generated (voltage generator enabled); note 1	6.0	–	8.5	V
I_{DD1}	supply current 1 (normal mode) for internal V_{LCD}	$V_{DD} = 2.85$ V; $V_{LCD} = 7.0$ V; $f_{SCLK} = 0$; $T_{amb} = 25$ °C; display load = 10 μ A; note 2	–	240	300	μ A
I_{DD2}	supply current 2 (normal mode) for internal V_{LCD}	$V_{DD} = 2.70$ V; $V_{LCD} = 7.0$ V; $f_{SCLK} = 0$; $T_{amb} = 25$ °C; display load = 10 μ A; note 2	–	–	320	μ A
I_{DD3}	supply current 3 (Power-down mode)	with internal or external LCD supply voltage; note 3	–	1.5	–	μ A
I_{DD4}	supply current external V_{LCD}	$V_{DD} = 2.85$ V; $V_{LCD} = 9.0$ V; $f_{SCLK} = 0$; notes 2 and 4	–	25	–	μ A
I_{LCD}	supply current external V_{LCD}	$V_{DD} = 2.7$ V; $V_{LCD} = 7.0$ V; $f_{SCLK} = 0$; $T = 25$ °C; display load = 10 μ A; notes 2 and 4	–	42	–	μ A
Logic						
V_{IL}	LOW level input voltage		V_{SS}	–	$0.3V_{DD}$	V
V_{IH}	HIGH level input voltage		$0.7V_{DD}$	–	V_{DD}	V
I_L	leakage current	$V_i = V_{DD}$ or V_{SS}	–1	–	+1	μ A
Column and row outputs						
$R_{O(C)}$	column output resistance C0 to C83		–	12	20	k Ω
$R_{O(R)}$	row output resistance R0 to R47		–	12	20	k Ω
$V_{bias(tol)}$	bias voltage tolerance on C0 to C83 and R0 to R47		–100	0	+100	mV

Figura 23: DC CHARACTERISTICS

SYMBOL	PARAMETER	CONDITIONS	MIN.	TYP.	MAX.	UNIT
LCD supply voltage generator						
V _{LCD}	V _{LCD} tolerance internally generated	V _{DD} = 2.85 V; V _{LCD} = 7.0 V; f _{SCLK} = 0; display load = 10 μ A; note 5	–	0	300	mV
TC0	V _{LCD} temperature coefficient 0	V _{DD} = 2.85 V; V _{LCD} = 7.0 V; f _{SCLK} = 0; display load = 10 μ A	–	1	–	mV/K
TC1	V _{LCD} temperature coefficient 1	V _{DD} = 2.85 V; V _{LCD} = 7.0 V; f _{SCLK} = 0; display load = 10 μ A	–	9	–	mV/K
TC2	V _{LCD} temperature coefficient 2	V _{DD} = 2.85 V; V _{LCD} = 7.0 V; f _{SCLK} = 0; display load = 10 μ A	–	17	–	mV/K
TC3	V _{LCD} temperature coefficient 3	V _{DD} = 2.85 V; V _{LCD} = 7.0 V; f _{SCLK} = 0; display load = 10 μ A	–	24	–	mV/K

Figura 24: DC CHARACTERISTICS

SYMBOL	PARAMETER	CONDITIONS	MIN.	TYP.	MAX.	UNIT
f _{OSC}	oscillator frequency		20	34	65	kHz
f _{clk(ext)}	external clock frequency		10	32	100	kHz
f _{frame}	frame frequency	f _{OSC} or f _{clk(ext)} = 32 kHz; note 1	–	67	–	Hz
t _{VHRL}	V _{DD} to RES LOW	Fig.16	0 ⁽²⁾	–	30	ms
t _{WL(RES)}	RES LOW pulse width	Fig.16	100	–	–	ns
Serial bus timing characteristics						
f _{SCLK}	clock frequency	V _{DD} = 3.0 V $\pm$ 10%	0	–	4.00	MHz
T _{cy}	clock cycle SCLK	All signal timing is based on 20% to 80% of V _{DD} and maximum rise and fall times of 10 ns	250	–	–	ns
t _{WH1}	SCLK pulse width HIGH		100	–	–	ns
t _{WL1}	SCLK pulse width LOW		100	–	–	ns
t _{su2}	$\overline{\text{SCE}}$ set-up time		60	–	–	ns
t _{h2}	$\overline{\text{SCE}}$ hold time		100	–	–	ns
t _{WH2}	$\overline{\text{SCE}}$ min. HIGH time		100	–	–	ns
t _{h5}	$\overline{\text{SCE}}$ start hold time; note 3		100	–	–	ns
t _{su3}	D/ $\overline{\text{C}}$ set-up time		100	–	–	ns
t _{h3}	D/ $\overline{\text{C}}$ hold time		100	–	–	ns
t _{su4}	SDIN set-up time		100	–	–	ns
t _{h4}	SDIN hold time		100	–	–	ns

Figura 25: AC CHARACTERISTICS

2.4. Comunicación UART/USART

La comunicación **UART** (Universal Asynchronous Receiver-Transmitter) es una tecnología utilizada para transmitir datos en serie, sin necesidad de una señal de reloj compartida entre los dispositivos. UART permite la comunicación entre dos dispositivos electrónicos (como microcontroladores, computadoras, o módulos Bluetooth) utilizando solo dos líneas: **TX** (transmisión) y **RX** (recepción).

El **USART** (Universal Synchronous and Asynchronous Receiver-Transmitter) es una versión más avanzada de UART, capaz de operar tanto en modo asíncrono, como UART, como en modo síncrono, donde sí se emplea una señal de reloj para sincronizar la comunicación.

Características principales:

- Modo de operación asíncrono (UART): no requiere una señal de reloj. Se basa en la configuración previa del baud rate (velocidad de transmisión) entre los dispositivos.
- Transmisión en serie: los datos se envían bit a bit, comenzando con un bit de inicio (start bit), seguido por los bits de datos, un bit opcional de paridad y uno o más bits de parada (stop bits).
- Comunicación bidireccional: permite el envío y recepción simultánea de datos (full-duplex).
- Simplicidad de conexión: sólo requiere dos líneas de datos (TX y RX).
- Baud rate común: valores como 9600, 115200, etc., deben ser iguales en ambos extremos para evitar errores.

2.5. Protocolo de Comunicación SPI (Serial Peripheral Interface)

El **SPI** (Interfaz Periférica Serial) es un protocolo de comunicación serial síncrono desarrollado por Motorola, utilizado para comunicar un microcontrolador con uno o varios dispositivos periféricos. A diferencia de UART, SPI sí utiliza una señal de reloj generada por el maestro para sincronizar el envío y recepción de datos.

2.5.1. Arquitectura del SPI

SPI se basa en una arquitectura maestro-esclavo, donde el maestro (generalmente el microcontrolador) controla el flujo de datos. Se requieren cuatro líneas principales para la comunicación:

- MOSI (Master Out Slave In): línea de datos enviada desde el maestro al esclavo.
- MISO (Master In Slave Out): línea de datos enviada desde el esclavo al maestro.
- SCLK (Serial Clock): señal de reloj generada por el maestro.
- SS / CS (Slave Select o Chip Select): línea usada para seleccionar qué esclavo está activo.

Características principales:

- Comunicación síncrona: requiere reloj compartido (SCLK).
- Alta velocidad de transferencia: más rápido que UART y I2C.
- Permite múltiples esclavos: a través de pines CS separados.
- Full-duplex: permite envío y recepción de datos al mismo tiempo.
- No incluye direccionamiento: cada esclavo debe ser gestionado manualmente por el CS.

2.6. Diseño del circuito

A continuación se presenta el diseño final del circuito:

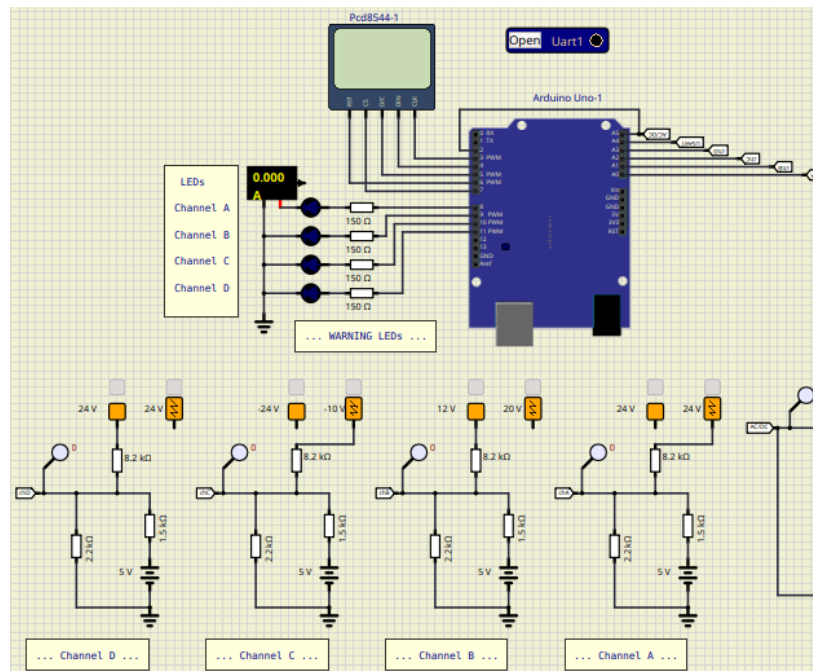


Figura 26: Diseño final, parte 1

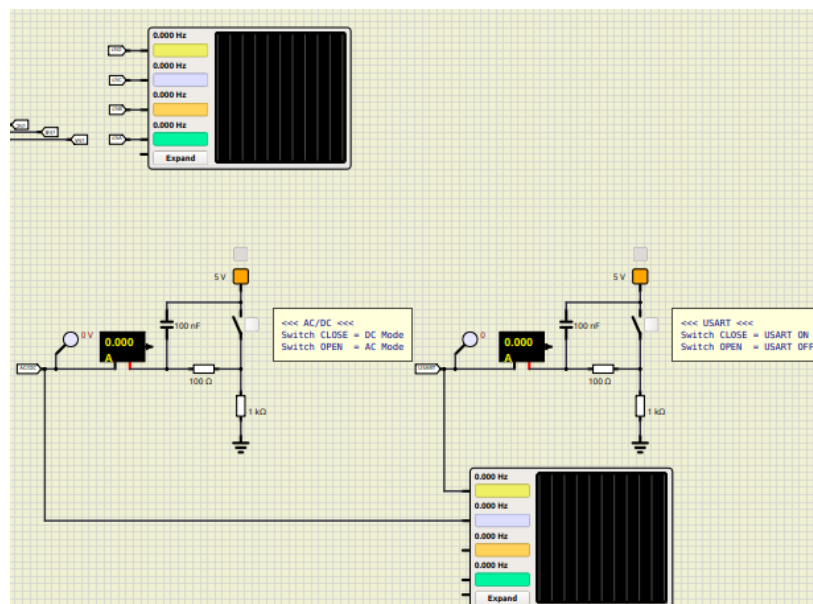


Figura 27: Diseño final, parte 2

Tabla 1: Lista de componentes

Componente	Cantidad	Precio (CRC)
Arduino UNO	1	-
PCD8544	1	-
1k Ω	2	-
100 Ω	2	-
100nF	2	-
150 Ω	4	-
8.2k Ω	4	-
2.2k Ω	4	-
1.5k Ω	4	-
Led rojo	4	-

2.6.1. Lista de componentes

2.6.2. Información adicional

GPIOs en Arduino

Los GPIOs (General Purpose Input/Output) son pines digitales que pueden ser configurados por el usuario como entradas o salidas, según se necesite. En el Arduino Uno, hay 14 pines digitales numerados del 0 al 13 y 6 pines analógicos (A0–A5) que también pueden funcionar como GPIOs digitales.

- Modo de entrada (INPUT): El pin puede leer el estado lógico (ALTO o BAJO) de un botón, sensor u otro dispositivo.
- Modo de salida (OUTPUT): El pin puede enviar señales digitales (HIGH = 5V, LOW = 0V) para controlar LEDs, relés, motores, etc.
- Modo INPUT_PULLUP: Activa una resistencia pull-up interna de 20–50 k Ω , útil para entradas con botones sin necesidad de resistencias externas.

Funciones comunes:

- `pinMode(pin, modo)` → configura el modo del pin.
- `digitalWrite(pin, valor)` → cambia el valor de salida.
- `digitalRead(pin)` → lee el valor de entrada.

Interrupciones en Arduino

Las interrupciones permiten que el microcontrolador reaccione inmediatamente a ciertos eventos externos, sin tener que esperar a que termine el ciclo principal del programa. En lugar de estar revisando constantemente si un evento ocurrió (polling), las interrupciones lo notifican de forma automática.

Arduino Uno admite interrupciones externas en los pines digitales 2 y 3 (INT0 e INT1).

Tipos de eventos que pueden generar una interrupción:

- Cambio de estado lógico (subida, bajada o cambio).
- Eventos de temporizador o comunicación (más avanzados).

```
attachInterrupt(digitalPinToInterrupt(pin), ISR, modo);
```

- `pin`: número de pin (ej. 2 o 3).
- `ISR`: nombre de la función que se ejecutará (debe ser rápida y sin `delay()`).
- `modo`: puede ser `RISING`, `FALLING`, `CHANGE`, `LOW`.

ejemplo:

```
attachInterrupt(digitalPinToInterrupt(2), encenderAlarma, RISING);
```

Convertidor Analógico-Digital (ADC)

Un ADC es un dispositivo electrónico que convierte una señal analógica continua (como una señal de voltaje) en una representación digital. Se usa ampliamente en electrónica digital para interactuar con el mundo físico, donde la mayoría de las señales son analógicas.

Proceso básico del ADC:

- Muestreo: se toma una muestra del valor de voltaje en un instante específico.
- Cuantificación: se asigna un valor numérico dentro de un rango discreto.
- Codificación: se genera el equivalente binario que representa esa muestra.

Parámetros importantes:

- Resolución: Cuántos niveles digitales puede representar (ej. 10 bits = 1024 niveles).
- Tasa de muestreo: Cuántas veces por segundo se mide la señal.
- Voltaje de referencia: El nivel máximo que puede leer el ADC.

Características del ADC en el Arduino Uno (ATmega328P):

- Resolución de 10 bits, lo que significa que convierte la señal analógica en valores entre 0 y 1023.
- Puede leer hasta 6 canales analógicos (A0 a A5).
- Tiempo de conversión aproximado: 13 ciclos del ADC por muestra.
- El voltaje de referencia puede ser externo o interno (por defecto 5V del sistema).

Función clave:

```
analogRead(pin); // Devuelve un valor entre 0 y 1023
```

3. Desarrollo

3.1. Análisis del firmware

A continuación, se realiza una descripción detallada y estructurada del funcionamiento del programa desarrollado para el laboratorio, cuyo propósito es medir voltajes en cuatro canales analógicos utilizando un Arduino Uno, mostrar los resultados en una pantalla PCD8544 y registrar los datos mediante comunicación serial.

3.1.1. Inclusión de Librerías y Declaración de Variables

```
#include <PCD8544.h>
static PCD8544 lcd;
```

Se incluye la librería `PCD8544.h` para controlar la pantalla LCD basada en el controlador Nokia 5110. Se crea una instancia estática del objeto `lcd`.

```
constexpr float maxInputVoltage = 24.0;
constexpr float adcMidpoint = 511.5;
constexpr int numChannels = 4;
constexpr int analogPins[numChannels] = {A0, A1, A2, A3};
constexpr int warningPins[numChannels] = {8, 9, 10, 11};
float voltages[numChannels] = {0.0};
```

Se definen constantes globales:

- `maxInputVoltage`: Voltaje máximo esperado del sistema.
- `adcMidpoint`: Punto medio del ADC para representar 0 V cuando se miden señales bipolares.
- `numChannels`: Número total de canales de medición.
- `analogPins` y `warningPins`: Arreglos que contienen los pines analógicos y pines para LEDs de advertencia, respectivamente.
- `voltages`: Arreglo donde se almacenan los valores medidos por canal.

3.1.2. Función `setup()`

```
void setup() {
    Serial.begin(9600);
    for (int pin : warningPins) {
        pinMode(pin, OUTPUT);
    }
    lcd.begin(84, 48);
    lcd.clear();
}
```

```
...
attachInterrupt(digitalPinToInterrupt(2), clear_display, CHANGE);
delay(5000);
}
```

- Se inicializa la comunicación serial.
- Se configuran los pines de advertencia como salidas.
- Se inicializa y configura la pantalla LCD, mostrando un mensaje de bienvenida.
- Se establece una interrupción externa en el pin digital 2, la cual limpia la pantalla LCD al detectar un cambio de estado.

3.1.3. Función de Interrupción: `clear_display()`

```
void clear_display() {
    lcd.clear();
}
```

Esta función se invoca automáticamente mediante una interrupción para limpiar la pantalla cuando se presiona un botón conectado al pin 2.

3.1.4. Funciones de Visualización

- Serial:

```
void serial_display(const char *mode, const float v[]) {
    if (analogRead(A4) > adcMidpoint) {
        Serial.println(mode);
        for (int i = 0; i < numChannels; ++i) {
            Serial.println(v[i]);
        }
    }
}
```

Solo envía los datos por el puerto serial si el pin A4 está por encima del umbral, usado como condición habilitadora.

- LCD – AC y DC:

```
void lcd_display_ACMODE(const char *mode, const float v[]) { ... }
void lcd_display_DCMODE(const char *mode, const float v[]) { ... }
```

Ambas funciones muestran los valores de voltaje en la pantalla LCD con etiquetas adecuadas (ej: (**V_{rms}**) para AC y (**V**) para DC), indicando el canal correspondiente.

3.1.5. Modo de Medición AC

```
float get_RMS(int pin) { ... }

void AC_mode() {
    for (int i = 0; i < numChannels; ++i) {
        voltages[i] = get_RMS(analogPins[i]);
    }
    ...
}
```

- `get_RMS()`: Calcula el valor eficaz (RMS) de una señal simulando una medición de voltaje pico a partir de múltiples muestras.
- `AC_mode()`: Calcula y ajusta los valores RMS, verifica sobrevoltajes, muestra en LCD y envía por serial.

3.1.6. Análisis detallado de la función `get_RMS()`

```
float get_RMS(int pin) {
    constexpr int samples = 1000;
    float tmax = 0.0;
    for (int i = 0; i < samples; ++i) {
        float t = analogRead(pin);
        if (t > tmax) tmax = t;
    }
    float peakVoltage = (tmax - adcMidpoint) * (maxInputVoltage / adcMidpoint);
    return peakVoltage * 0.7071f; // Aproximación de RMS
}
```

Esta función tiene como propósito calcular una aproximación del valor eficaz (RMS) de una señal de voltaje alterna (AC) leída por un canal analógico del Arduino Uno. El valor RMS (Root Mean Square) es fundamental en sistemas de medición porque indica el valor de voltaje que produciría el mismo efecto térmico que un voltaje DC constante.

- Número de muestras

```
constexpr int samples = 1000;
```

- Se toman 1000 muestras del pin analógico para tener una estimación estable del valor pico de la señal.
- Esta cantidad es una buena elección para una señal periódica como el seno, pero es arbitraria y puede ajustarse según el tiempo de adquisición o la resolución deseada.

■ Búsqueda del valor máximo (pico)

```
float tmax = 0.0;
for (int i = 0; i < samples; ++i) {
    float t = analogRead(pin);
    if (t > tmax) tmax = t;
}
```

- Se escanea la señal buscando el mayor valor leído por el ADC, es decir, el valor pico (positivo) de la señal.
- `analogRead()` devuelve un valor entre 0 y 1023 correspondiente a un voltaje entre 0V y 5V.
- Para una señal alterna centrada en el punto medio del ADC (511.5), el valor pico se encontrará por encima de esa referencia.

■ Conversión de valor pico en voltios

```
float peakVoltage = (tmax - adcMidpoint) * (maxInputVoltage / adcMidpoint);
```

- `tmax - adcMidpoint`: representa cuán por encima del punto medio está el valor pico (en términos de pasos del ADC).
- `(maxInputVoltage / adcMidpoint)`: factor de escala para convertir pasos del ADC en voltios reales. Este factor surge del hecho de que el ADC va de 0 a 1023, por lo tanto, su punto medio es 511.5 (asumiendo señales bipolarizadas entre -24V y +24V).
- El resultado de esta operación es el valor pico real de la señal en voltios.

■ Cálculo del valor RMS

```
return peakVoltage * 0.7071f;
```

- Aquí se usa la fórmula:

$$V_{RMS} = \frac{V_{pico}}{\sqrt{2}} \approx V_{pico} \cdot 0,7071$$

- Esta fórmula solo es válida para señales senoidales puras.
- Si la señal es distorsionada o de otra forma (triangular, cuadrada), esta fórmula no es precisa.

3.1.7. Modo de Medición DC

```
void DC_mode() {  
    for (int i = 0; i < numChannels; ++i) {  
        voltages[i] = (analogRead(analogPins[i]) - adcMidpoint) * (maxInputVoltage / adcMidpoint);  
    }  
    ...  
}
```

- Se calcula el voltaje en DC ajustando la lectura respecto al punto medio del ADC.
- Se verifica advertencia de sobrevoltaje y se muestra en LCD y puerto serial.

3.1.8. Advertencia de Alto Voltaje

```
void warning_leds(const float v[]) {  
    for (int i = 0; i < numChannels; ++i) {  
        digitalWrite(warningPins[i], (v[i] > 20.0 || v[i] < -20.0) ? HIGH : LOW);  
    }  
}
```

- Enciende un LED por canal si el voltaje medido está fuera del rango seguro ± 20 V.

3.1.9. Selector de Modo

```
void Switch_mode() {  
    (analogRead(A5) < adcMidpoint) ? AC_mode() : DC_mode();  
}
```

- El sistema detecta el modo AC o DC según el voltaje leído en el pin A5 (usualmente conectado a un botón o divisor de voltaje).

3.1.10. Bucle Principal

```
void loop() {  
    Switch_mode();  
}
```

- El sistema permanece en un bucle continuo, midiendo voltajes en modo AC o DC según el valor leído en el pin A5.

3.2. Análisis electrónico

3.2.1. Conversión de rango de tensión: De [-24V, +24V] a [0V, +5V]

Dado que el Arduino Uno solo puede medir tensiones en el rango de 0V a 5V, es necesario adaptar la señal de entrada para que se encuentre dentro de ese intervalo. Para ello, se emplea un circuito divisor de tensión con doble fuente, como se muestra en al siguiente Figura:

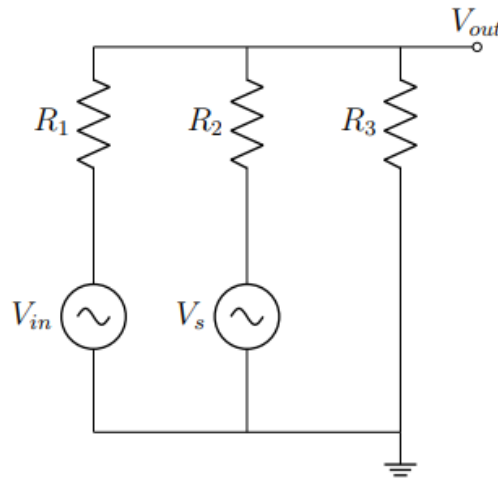


Figura 28: Divisor de tensión con dos fuentes

El análisis del circuito se realiza mediante el teorema de superposición, el cual establece que, para un circuito lineal con múltiples fuentes, la respuesta total en cualquier elemento es igual a la suma de las respuestas individuales generadas por cada fuente actuando por separado (mientras las demás están desactivadas).

Primero se analiza el efecto de la fuente de alimentación V_s , mientras que la fuente de entrada V_{in} se reemplaza por un cortocircuito (fuente de tensión apagada). Aplicando ley de Ohm y reglas de división de tensión se obtiene:

$$V_{OUT1} = \frac{R_1 || R_3}{(R_1 || R_3) + R_2} \cdot V_s \quad (1)$$

Luego, se analiza el efecto de la fuente de entrada V_{in} , desactivando V_s . El análisis es análogo y resulta en:

$$V_{OUT2} = \frac{R_2 || R_3}{(R_2 || R_3) + R_1} \cdot V_{in} \quad (2)$$

La tensión de salida total V_{OUT} es la suma de ambos aportes

$$V_{OUT} = V_{OUT1} + V_{OUT2} \quad (3)$$

$$\boxed{\frac{R_1 || R_3}{(R_1 || R_3) + R_2} \cdot V_s + \frac{R_2 || R_3}{(R_2 || R_3) + R_1} \cdot V_{in}} \quad (4)$$

Este sistema ofrece cuatro grados de libertad (las resistencias R_1 , R_2 , R_3 , R_4 y la fuente de referencia V_S).

Sabemos que:

- Si $V_{in} = +24$, entonces $V_{OUT} = 5V$
- Si $V_{in} = -24$, entonces $V_{OUT} = 0V$

Estas condiciones representan dos ecuaciones, por lo tanto, es posible seleccionar dos variables libremente, se va a utilizar $V_S = 5V$.

$$5 = \frac{R_1 || R_3}{(R_1 || R_3) + R_2} \cdot 5 + \frac{R_2 || R_3}{(R_2 || R_3) + R_1} \cdot 24 \quad (5)$$

$$0 = \frac{R_1 || R_3}{(R_1 || R_3) + R_2} \cdot 5 + \frac{R_2 || R_3}{R_2 || R_3} \cdot -24 \quad (6)$$

Resolviendo para obtener con valores de resistencias comerciales, se tiene que:

$$R_1 = 8,2k\Omega \quad (7)$$

$$R_2 = 1,5k\Omega \quad (8)$$

$$R_3 = 2,2k\Omega \quad (9)$$

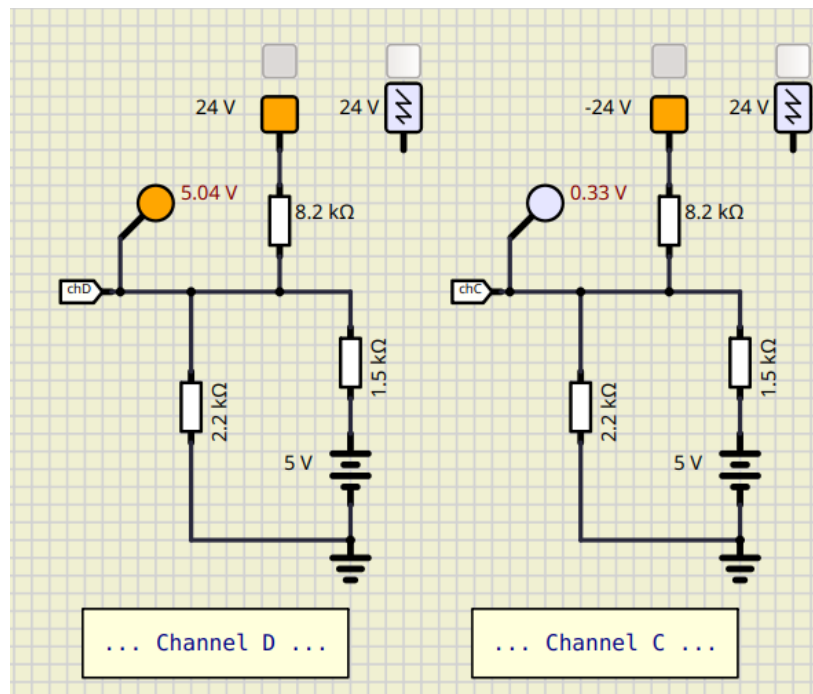


Figura 29: Conversión de rango de tensión

Como se observa los valores de tensión no son exactamente los esperados pero esto se debe al uso de resistencias comerciales en su totalidad, por lo que la aproximación es válida para efectos de trabajo.

3.2.2. Implementación de los interruptores (Switches)

Para la activación del modo AC/DC y la transmisión por USART, se implementaron interruptores (switches) conectados a las entradas analógicas del Arduino Uno. Estos switches utilizan una configuración pull-down para asegurar una lectura estable y definida en estado bajo. La configuración incluye una resistencia de $1k\Omega$ con un capacitor de $100nF$, formando un filtro RC que contribuye a reducir rebotes mecánicos.

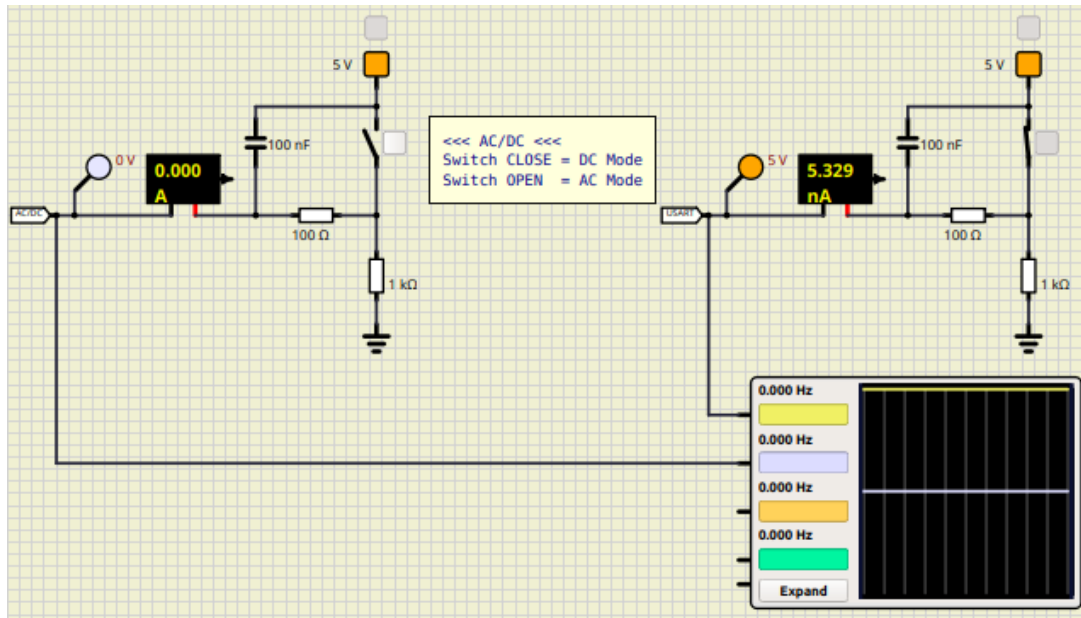


Figura 30: Interruptores

La constante de tiempo del filtro se calcula como:

$$\tau = R \cdot C = 1k\Omega \cdot 100nF = 100\mu s$$

Esto indica que el sistema tarda aproximadamente $100\mu s$ en responder a un cambio de estado, lo cual es suficiente para eliminar rebotes de corta duración sin afectar la capacidad de detección del pulsador. En caso de necesitar una mayor supresión de rebotes, podría incrementarse la resistencia o el valor del capacitor para alcanzar una constante de tiempo mayor (por ejemplo, $1ms$).

3.2.3. Iluminación de los Leds

En la siguiente Figura se observa el uso de un amperímetro conectado en serie con uno de los LEDs de advertencia. Los LEDs utilizados son de color rojo, con un umbral de encendido de aproximadamente $2V$ y una resistencia limitadora de 150Ω . Se registra una corriente de 15.74 mA , valor coherente con los parámetros de diseño y dentro del rango seguro de funcionamiento para este tipo de componentes, lo que demuestra que el sistema activa correctamente los avisos cuando la tensión supera los umbrales establecidos.

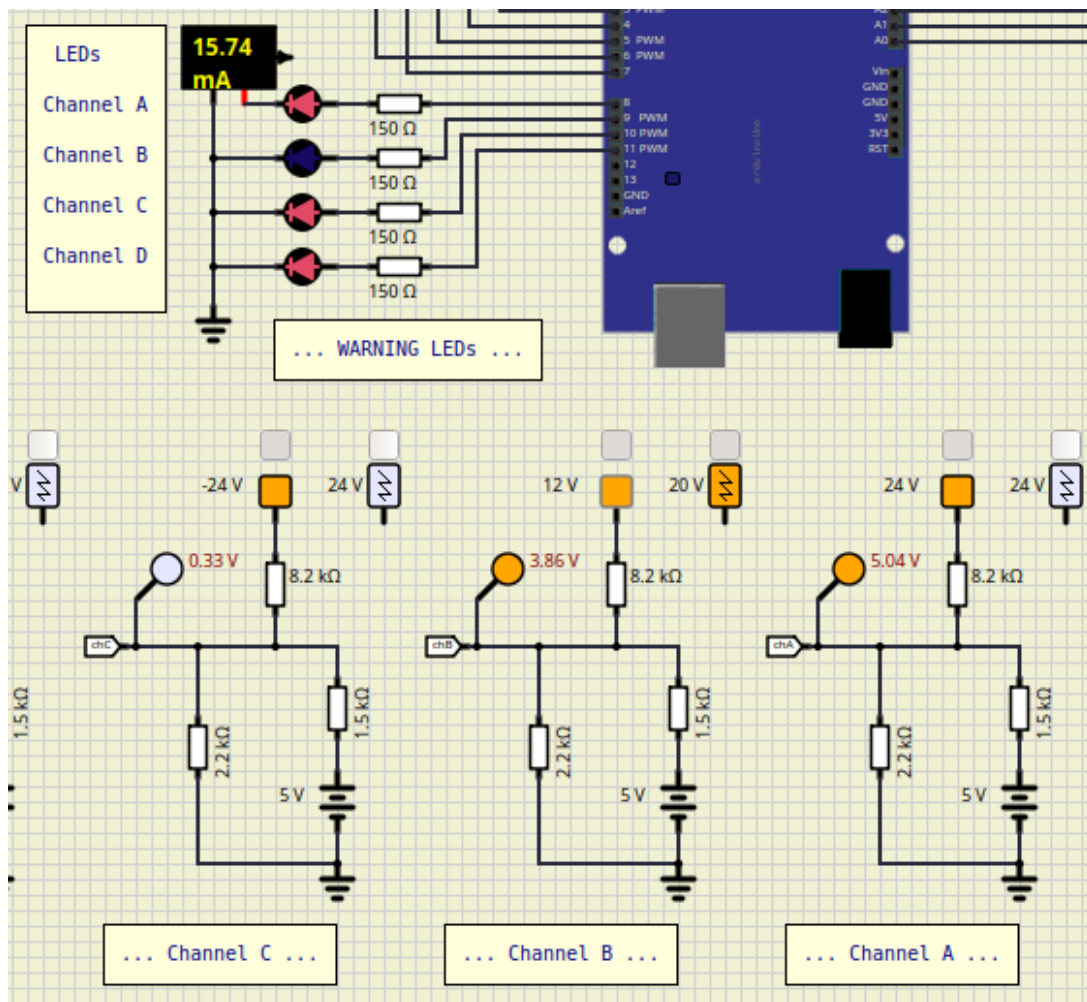


Figura 31: Led de advertencia

3.2.4. Pantalla PCD8544

La Figura muestra el mensaje de bienvenida desplegado en la pantalla LCD al iniciar el sistema. Este mensaje es una confirmación visual de que la pantalla y la comunicación con el microcontrolador funcionan adecuadamente, lo que también indica que la inicialización del sistema se realiza de forma correcta.

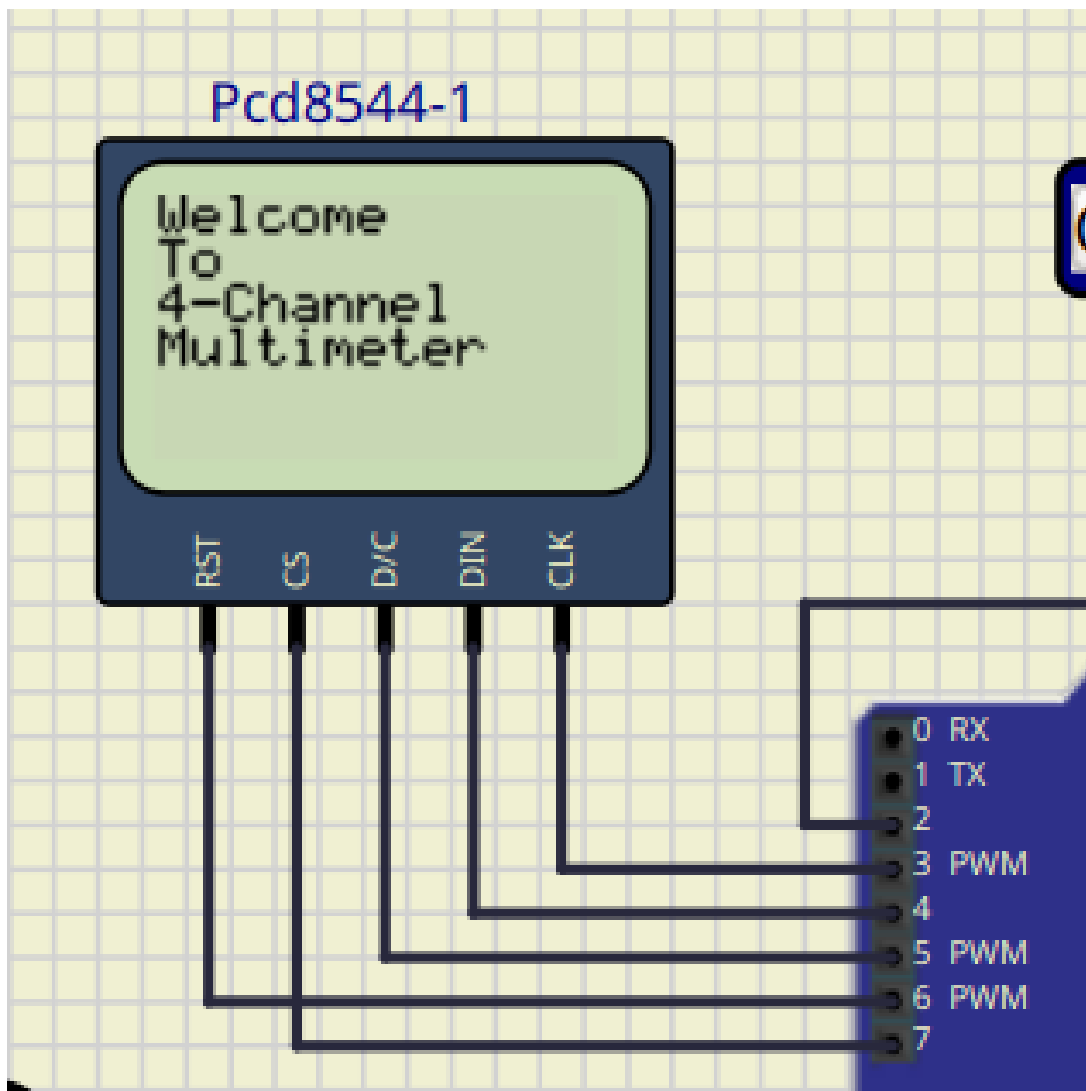


Figura 32: Funcionamiento de la pantalla PCD8544

3.2.5. Señales de los 4 canales

En la siguiente Figura se visualiza el comportamiento de las señales medidas en los distintos canales. Dos canales están configurados para visualizar tensiones DC, mientras que los otros dos muestran señales AC. Las lecturas y visualización en pantalla permiten comprobar que el sistema identifica correctamente el tipo de señal y realiza el procesamiento adecuado para cada modo, aplicando el cálculo RMS en el caso del AC y ajustando la escala en el caso del DC.

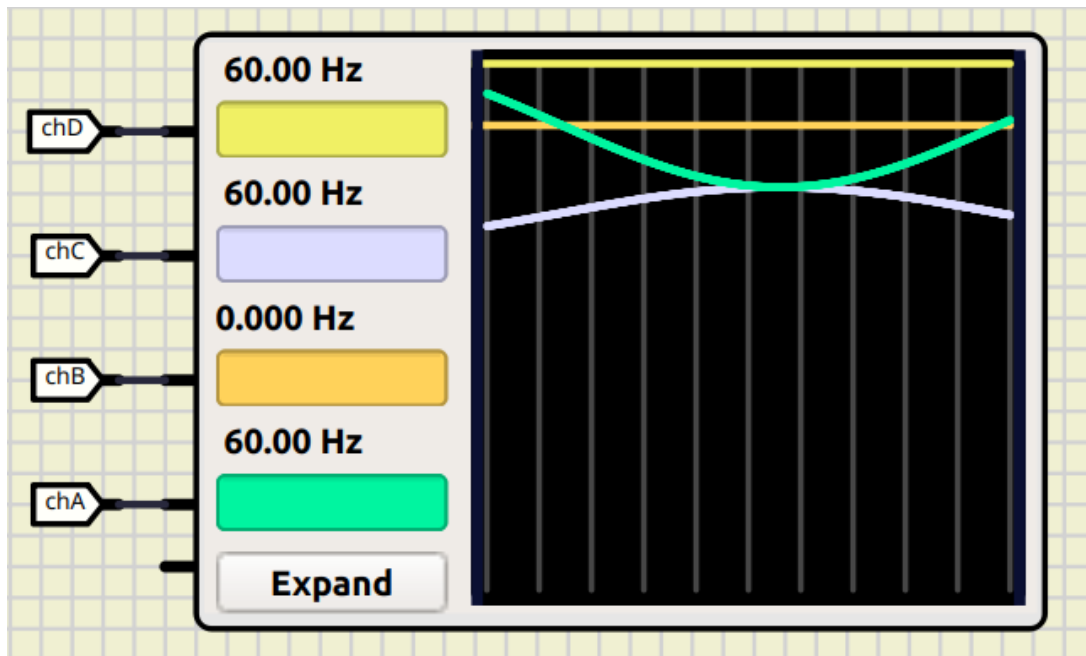


Figura 33: Visualización de las señales a través del osciloscopio

3.3. Análisis de resultados

3.3.1. Análisis en modo AC

Durante la simulación en modo AC se observaron los resultados del sistema cuando los canales de entrada recibieron señales alternas. En este modo, el microcontrolador ejecuta la rutina `get_RMS()`, la cual obtiene el valor pico de una señal (entre 1000 muestras) y lo convierte en un valor RMS utilizando la fórmula correspondiente.

En las Figuras se aprecia cómo los 4 canales fueron configurados para trabajar con señales AC. Los valores RMS se calcularon correctamente y se desplegaron en pantalla con el sufijo Vrms, indicando con claridad que se trataba de señales alternas. El sistema mostró una respuesta estable y precisa ante entradas alternas simuladas, cumpliendo su objetivo de ofrecer lecturas confiables para señales AC.

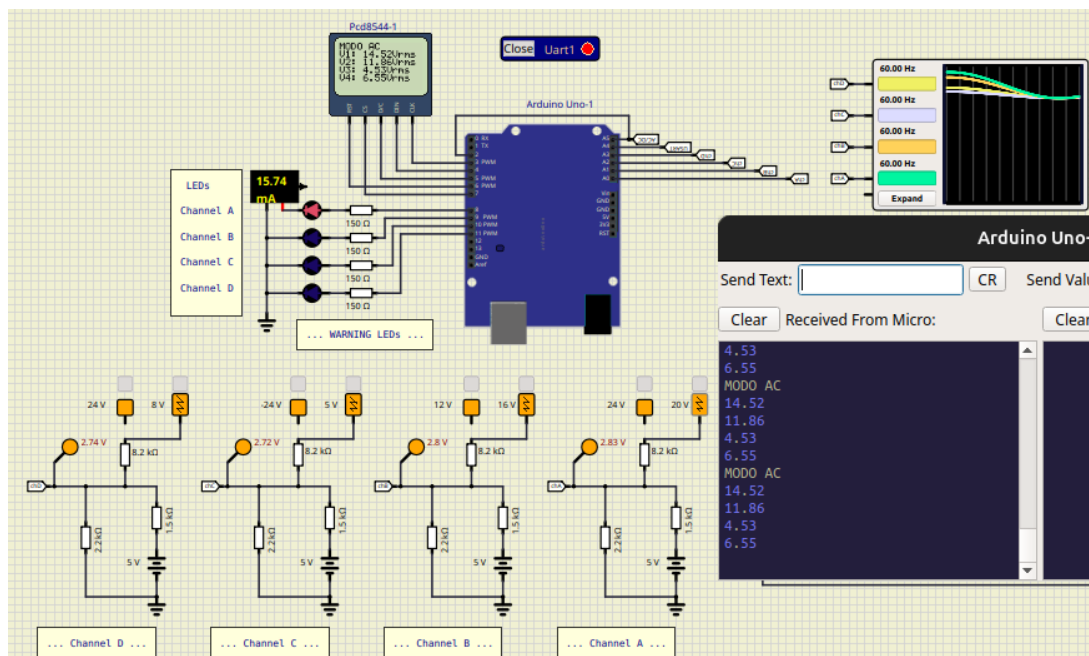


Figura 34: Resultados de las mediciones AC.

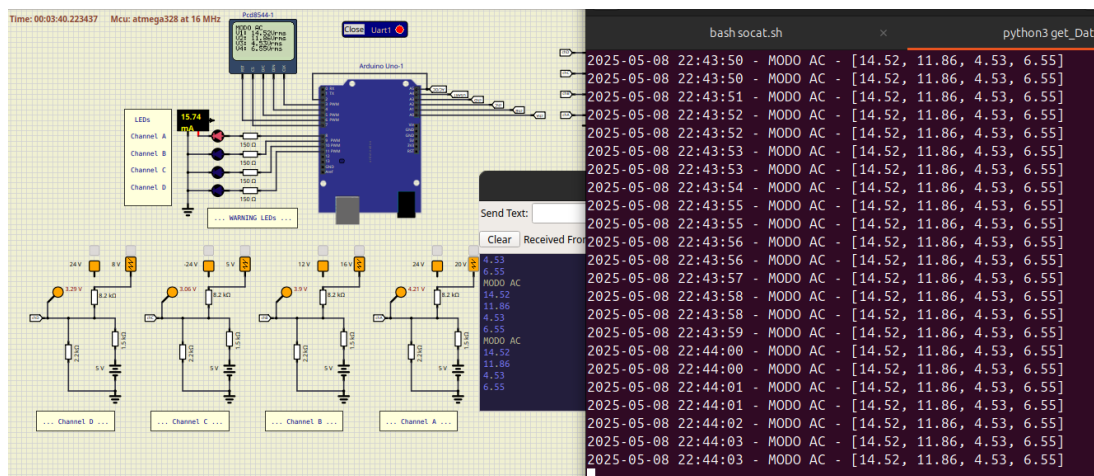


Figura 35: Funcionamiento en modo AC y USART activado

3.3.2. Análisis en modo DC

En el modo DC, el sistema mide directamente la diferencia entre el valor leído por el ADC y el punto medio (511.5), que representa 0V, y luego lo convierte en un valor dentro del rango de $[-24V, +24V]$. Este modo se activa automáticamente al detectar que el pin de control A5 supera el umbral establecido.

Durante la simulación, se activan los canales con señales de corriente continua (DC). En las Figuras correspondientes, se muestran cómo los valores de tensión fueron procesados correctamente y se desplegaron en pantalla con el sufijo V, diferenciándolos de las mediciones en modo AC.

Adicionalmente, el sistema de advertencia con LEDs rojos funcionó correctamente. Para tensiones mayores a $\pm 20V$, se encendieron los LEDs correspondientes. Esto fue confirmado mediante un amperímetro conectado al circuito, el cual mostró un paso de corriente cercano a los 15.74mA, lo que concuerda con una caída de tensión de aproximadamente 2V en el LED, y una resistencia limitadora de 150Ω , cumpliendo con lo esperado teóricamente por la ley de Ohm.

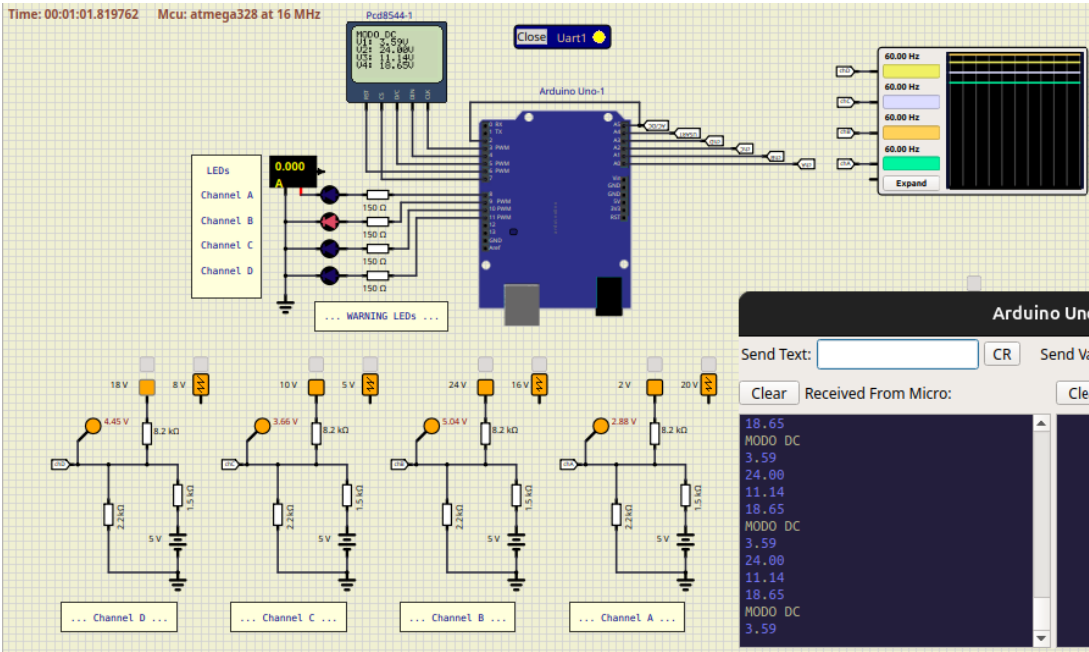


Figura 36: Resultados de las mediciones DC.

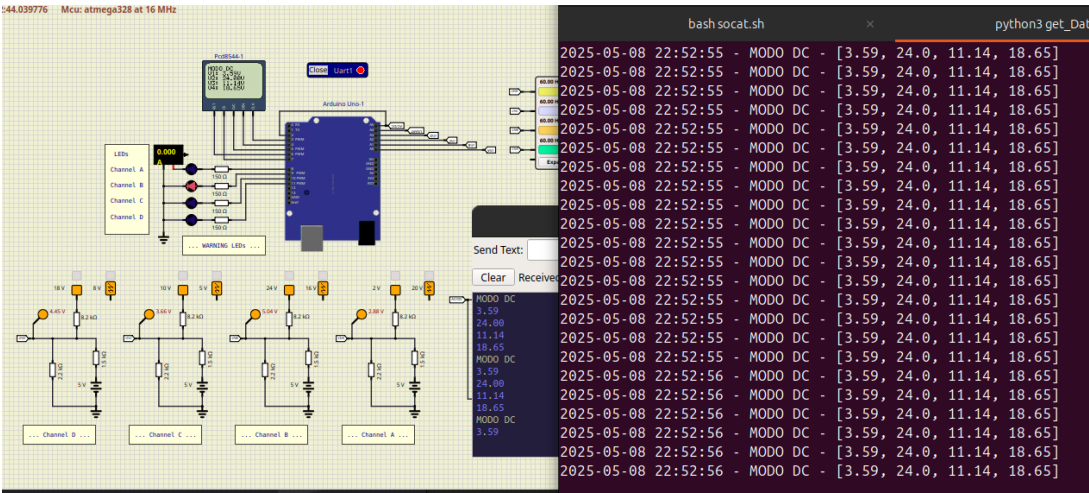


Figura 37: Funcionamiento en modo DC y USART activado

3.3.3. Archivo .csv

	A	B	C	D	E	F	G
352	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
353	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
354	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
355	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
356	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
357	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
358	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
359	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
360	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
361	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
362	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
363	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
364	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
365	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
366	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
367	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
368	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
369	2025-05-08 22:43:38	MODQ AC	15.21	11.86	1.21	6.55	
370	2025-05-08 22:43:38	MODQ DC	19.96	15.41	-10.91	8.33	
371	2025-05-08 22:43:38	MODQ DC	2.13	2.13	1.15	2.09	
372	2025-05-08 22:43:38	MODQ DC	15.13	12.27	-8.47	7.3	
373	2025-05-08 22:43:38	MODQ DC	13.77	10.53	-6.22	5.75	
374	2025-05-08 22:43:38	MODQ DC	4.53	4.11	-0.77	3.21	
375	2025-05-08 22:43:38	MODQ DC	20.53	16.21	-11.99	9.08	
376	2025-05-08 22:43:38	MODQ DC	4.11	3.31	0.4	2.32	
377	2025-05-08 22:43:38	MODQ DC	5.65	5	-1.67	3.68	
378	2025-05-08 22:43:38	MODQ DC	21.04	16.54	-12.27	9.22	
379	2025-05-08 22:43:38	MODQ DC	3.26	2.7	0.91	2.04	
380	2025-05-08 22:43:38	MODQ DC	6.78	5.89	-2.51	4.15	
381	2025-05-08 22:43:38	MODQ DC	21.37	16.73	-12.41	9.27	
382	2025-05-08 22:43:38	MODQ DC	2.56	2.23	1.34	1.85	
383	2025-05-08 22:43:38	MODQ DC	8.23	7.01	-3.59	4.72	
384	2025-05-08 22:43:38	MODQ DC	21.51	16.77	-12.36	9.22	
385	2025-05-08 22:43:38	MODQ DC	2.04	1.9	1.62	1.76	
386	2025-05-08 22:43:38	MODQ DC	9.55	8.05	-4.57	5.23	
387	2025-05-08 22:43:38	MODQ DC	21.37	16.63	-12.22	9.08	
388	2025-05-08 22:43:38	MODQ DC	1.81	1.71	1.71	1.71	
389	2025-05-08 22:43:38	MODQ DC	11.1	9.22	-5.65	5.79	
390	2025-05-08 22:43:38	MODQ DC	19.68	15.18	-10.72	8.23	

Figura 38: Archivo .csv con las mediciones del multímetro

4. Conclusiones y Recomendaciones

4.1. Conclusiones

- El uso del ADC de 10 bits del Arduino Uno permitió una conversión adecuada de las señales analógicas en un rango amplio de voltaje mediante el uso de técnicas de acondicionamiento y escalado.
- Se logró implementar con éxito un sistema de medición multicanal con visualización gráfica simple, gracias a la pantalla PCD8544 y la biblioteca correspondiente.
- La distinción entre modo AC y DC fue fundamental para adaptarse a distintos tipos de señales, mostrando la importancia de la interpretación de valores RMS en señales alternas.
- La integración con Python mediante comunicación UART/USART y almacenamiento de datos en CSV permitió extender el alcance del proyecto hacia el registro y análisis de datos en la PC.

4.2. Recomendaciones

- Tener el código bien ordenado y comentado ayuda muchísimo, sobre todo en simulaciones donde todo depende de que entiendas bien cada parte. Nombrar bien las variables y separar claramente las funciones facilita la lectura y posibles cambios después.
- Es buena idea probar el sistema por partes antes de armar todo el código junto. Por ejemplo, primero verificar que las lecturas analógicas funcionan, luego probar la pantalla y por último integrar los modos AC/DC.
- Tener cuidado con el uso del ADC en simulación, ya que los valores pueden parecer erráticos si no se les da una señal consistente. A veces conviene simular voltajes estables o usar sliders en SimulIDE para controlar la entrada.
- Al trabajar con múltiples canales, ayuda mucho usar estructuras o arreglos, como se hizo en este caso. Eso hace el código más limpio y evita repetir líneas innecesarias.
- Probar bien las condiciones de los LEDs de advertencia, porque en simulación a veces no se nota si se activan o no. Conviene agregar mensajes seriales o probar con señales extremas para asegurarse que respondan correctamente.
- Por último, aunque sea una simulación, es bueno pensar cómo funcionaría esto en la vida real, por ejemplo, qué pasaría si se desconecta un sensor o si se quiere medir algo fuera del rango. Así se puede anticipar posibles mejoras.

Bibliografía

1. Material de clase
2. datasheet red_Led
3. Repositorio GitHub: <https://github.com/carlosefr/pcd8544>
4. Arduino. (n.d.). Arduino Uno Rev3. Recuperado de <https://store.arduino.cc/>
5. Atmel Corporation. (2016). ATmega328P Datasheet (Rev. 7810D–AVR–01/15). Recuperado de https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
6. SparkFun Electronics. (n.d.). Using the Nokia 5110 LCD with Arduino. Recuperado de <https://learn.sparkfun.com/tutorials/graphic-lcd-hookup-guide>
7. AllAboutCircuits. (n.d.). Root Mean Square (RMS) Voltage. Recuperado de <https://www.allaboutcircuits.com/tools/rms-voltage-calculator/>

5. Apéndices

5.1. Repositorio Git

Repositorio GitHub: <https://github.com/Rossetas/MCU/tree/lab3>

5.2. Archivo .sh

```
#!/bin/sh
```

```
socat PTY,link=/tmp/ttyS0,raw,echo=0 PTY,link=/tmp/ttyS1,raw,echo=0
```

5.3. Archivo .py

```
import serial
import csv
import time
from datetime import datetime

# Configuración del puerto serial
SERIAL_PORT = '/tmp/ttyS1'
BAUD_RATE = 9600

# Archivo CSV donde se guardan los datos
CSV_FILENAME = 'log.csv'

def main():
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        print(f"Conectado al puerto serial {SERIAL_PORT}")

        # Esperar
        time.sleep(2)

        with open(CSV_FILENAME, mode='w', newline='') as file:
            writer = csv.writer(file)
            # Encabezados
            writer.writerow(['Timestamp', 'Modo', 'V1', 'V2', 'V3', 'V4'])

        while True:
            # Leer modo
            mode = ser.readline().decode().strip()
            if mode.startswith("MOD0"):
                # Leer los siguientes 4 voltajes
                voltages = []
                for _ in range(4):
                    line = ser.readline().decode().strip()
                    try:
                        volt = float(line)
                    except ValueError:
                        volt = None
                    voltages.append(volt)

                # Registrar con timestamp
                timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
                writer.writerow([timestamp, mode] + voltages)
                print(f"{timestamp} - {mode} - {voltages}")

    except serial.SerialException as e:
```

```
        print(f"Error abriendo el puerto serial: {e}")
    except KeyboardInterrupt:
        print("END.")

if __name__ == "__main__":
    main()
```